DATABASE SPECIFICATION / DATA DICTIONARY

MEDSYS Database Dictionary

Detailed database reference for the MEDSYS backend, covering tables, columns, defaults, primary keys, foreign keys, uniqueness, nullability, indexes, check constraints, enums, and partitioning behavior.

DOCUMENT TYPE
As-built relational database dictionary

SYSTEM
MEDSYS API Enterprise Backend

VERSION
1.8

DATE
April 6, 2026

SOURCE OF TRUTH
Flyway migrations V1, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, and V17

AUDIENCE
Developers, QA, BA, reviewers, and client stakeholders

Prepared from the current repository schema migrations. This document describes the implemented database design, not a proposed future-state model.

1. Scope and Reading Guide

This document provides a field-level dictionary for the MEDSYS PostgreSQL schema. It focuses on database-enforced validation and structure: data type, required versus nullable, default values, generated values, primary keys, unique constraints, foreign keys, indexes, and check constraints.

> **Important:** application-level validations may be stricter than the database. This document records what the database itself enforces. Migration `V8` adds client/demo seed data, `V9` adds user-level extra permission storage, `V10` adds patient codes plus guardian identity fields, `V11` adds doctor workflow identity, `V12` adds persistent multi-role support, `V13` adds workflow timing fields, `V14` expands inventory metadata, `V15` adds doctor-facing inventory fields, `V16` adds inventory packaging-conversion fields, and `V17` adds batch-level inventory tracking and batch-linked movements.

2. Global Design Rules

- Most business tables are organization-scoped through `organization_id`.
- Most tables use surrogate numeric primary keys via `BIGSERIAL` or a sequence-backed `BIGINT`.
- Many business tables use soft deletion through nullable `deleted_at`.
- Appointments and audit logs are partitioned for operational scale.
- Authentication uses refresh-token family tracking for replay detection.

3. Enum Types

ENUM TYPE	ALLOWED VALUES	USED BY
user_role	`owner`, `doctor`, `assistant`	`users.role`, `users.active_role`, `user_roles.role`
gender	`male`, `female`, `other`	`patients.gender`
allergy_severity	`low`, `moderate`, `high`	`patient_allergies.severity`
appointment_status	`waiting`, `in_consultation`, `completed`, `cancelled`	`appointments.status`
priority_level	`low`, `normal`, `high`, `critical`	`appointments.priority`
test_order_status	`ordered`, `in_progress`, `completed`, `cancelled`	`test_orders.status`
drug_source	`clinical`, `outside`	`prescription_items.source`
inventory_category	`medicine`, `consumable`, `equipment`, `other`	`inventory_items.category`
inventory_movement_type	`in`, `out`, `adjustment`	`inventory_movements.movement_type`

4. Relationship Summary

PARENT TABLE	CHILD TABLE	RELATIONSHIP
users	`refresh_tokens`, `user_roles`	One user to many refresh tokens and one user to many assigned role rows.
families	`patients`, `family_members`	Families optionally attach directly to patients and also map through a join table.
patients	`family_members`, `patient_allergies`, `patient_conditions`, `appointments`, `encounters`, `prescriptions`, `patient_vitals`, `patient_timeline_events`, `patient_history_entries`	Patient is the central clinical entity and is referenced across most domain tables.
appointments	`encounters`	One encounter per appointment, enforced by unique `encounters.appointment_id` and a composite FK to partitioned

PARENT TABLE	CHILD TABLE	RELATIONSHIP
		appointments.
encounters	`encounter_diagnoses`, `test_orders`, `prescriptions`, `patient_vitals`, `patient_timeline_events`	Encounter is the anchor for diagnoses, test orders, prescriptions, and some clinical observations.
prescriptions	`prescription_items`, `dispense_records`	One prescription to many items and many dispense events.
inventory_items	`inventory_batches`, `inventory_movements`	One inventory item to many batch rows and many immutable movement rows.
inventory_batches	`inventory_movements`	One inventory batch can be referenced by many movement rows.
users	`appointments`, `encounters`, `prescriptions`, `dispense_records`, `inventory_movements`, `patient_history_entries`, `audit_logs`	Users appear as operational actors throughout the system.

5. Core Security and Identity Tables

5.1 users

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant/organization scope key.
`uuid`	UUID	Yes	`gen_random_uuid()`	Unique row UUID.
`email`	CITEXT	Yes	None	Case-insensitive email; unique with `organization_id`.
`password_hash`	TEXT	Yes	None	Hashed password only.
`first_name`	VARCHAR(80)	Yes	None	Required given name.
`last_name`	VARCHAR(80)	Yes	None	Required surname.
`role`	user_role	Yes	None	Legacy compatibility field aligned to the currently selected active role.
`active_role`	user_role	No	None	Persisted current workspace role for multi-role users; added in `V12`.
`doctor_workflow_mode`	doctor_workflow_mode	No	None	Explicit doctor business-mode identity. Set for doctor users and added in `V11`.
`extra_permissions`	JSONB	Yes	`'[]'::jsonb`	Stores per-user permission overrides; added in `V9`.
`is_active`	BOOLEAN	Yes	`TRUE`	Operational active flag.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.

Constraints: primary key on `id`; unique on `uuid`; unique on `(organization_id, email)`. **Notes:** `extra_permissions` is a required JSONB array column with an empty-array default after `V9`, `doctor_workflow_mode` was added in `V11`, and `active_role` was added in `V12` to support persisted workspace selection for multi-role users.

5.2 user_roles

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`user_id`	BIGINT	Yes	None	Foreign key to `users.id`.
`role`	user_role	Yes	None	Assigned role for the user.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.

Constraints: primary key on `id`; unique on `(user_id, role)`; foreign key `user_id -> users.id`. **Index:** `user_roles_user_idx`.

5.3 refresh_tokens

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`user_id`	BIGINT	Yes	None	Foreign key to `users.id`.
`token_id`	UUID	Yes	`gen_random_uuid()`	Unique token identifier.

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`family_id`	UUID	Yes	None at DB level	Refresh-token family identifier; added in V6.
`parent_token_id`	UUID	No	None	Logical previous token pointer; no FK constraint.
`expires_at`	TIMESTAMPTZ	Yes	None	Expiration timestamp.
`used_at`	TIMESTAMPTZ	No	None	Marked when refresh token is consumed.
`revoked_at`	TIMESTAMPTZ	No	None	Marked when revoked.
`replay_detected_at`	TIMESTAMPTZ	No	None	Set when replay is detected.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.

Constraints: primary key on `id`; unique on `token_id`; foreign key `user_id -> users.id`. **Indexes:** `refresh_tokens_user_idx`, `refresh_tokens_family_idx`.

6. Family and Patient Master Data

6.1 families

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`family_code`	VARCHAR(30)	Yes	None	Unique family code.
`family_name`	VARCHAR(120)	Yes	None	Required family label.
`assigned`	BOOLEAN	Yes	`FALSE`	Operational assignment flag.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Constraints: primary key on `id`; unique on `family_code`. **Index:** `families_org_assigned_idx`.

6.2 patients

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`uuid`	UUID	Yes	`gen_random_uuid()`	Unique patient UUID.
`patient_code`	VARCHAR(24)	Yes	Application generated	Always-present patient identifier added in `V10`; unique per organization.
`nic`	VARCHAR(20)	No	None	Optional identity number; unique per organization when present.
`first_name`	VARCHAR(80)	Yes	None	Required.
`last_name`	VARCHAR(80)	Yes	None	Required.
`full_name`	VARCHAR(170)	Yes	Generated stored	`first_name || ' ' || last_name`.
`dob`	DATE	Yes	None	Required date of birth; set `NOT NULL` in `V7`.
`age`	SMALLINT	Yes	None	Required explicit age snapshot; set `NOT NULL` in `V7`.
`gender`	gender	Yes	None	Enum constrained.

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`phone`	VARCHAR(30)	No	None	Optional phone.
`address`	TEXT	No	None	Optional address.
`blood_group`	VARCHAR(5)	No	None	Optional short blood group.
`family_id`	BIGINT	No	None	Optional foreign key to `families.id`.
`guardian_patient_id`	BIGINT	No	None	Optional link to another patient row used as guardian; added in `V10`.
`guardian_name`	VARCHAR(120)	No	None	Optional guardian display name for minors or dependent patients.
`guardian_nic`	VARCHAR(20)	No	None	Optional guardian NIC, searchable and not unique.
`guardian_phone`	VARCHAR(30)	No	None	Optional guardian phone, searchable and not unique.
`guardian_relationship`	VARCHAR(40)	No	None	Optional guardian relationship label such as `mother`, `father`, or `guardian`.
`is_active`	BOOLEAN	Yes	`TRUE`	Operational active flag.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Constraints: primary key on `id`; unique on `uuid`; unique `patients_org_nic_unique` on `(organization_id, nic)`; unique `patients_org_patient_code_unique` on `(organization_id, patient_code)`; foreign key `family_id -> families.id`; foreign key `guardian_patient_id -> patients.id`; `full_name`, `dob`, `age`, and `patient_code` are `NOT NULL` after `V10`. **Indexes:** `patients_org_full_name_idx`, `patients_org_family_idx`, `patients_org_patient_code_idx`, `patients_org_guardian_patient_idx`, `patients_org_guardian_nic_idx`.

Application behavior note: the backend can auto-create a related family row during patient registration when no `family_id` or family code is supplied. In that case the application generates a family label such as `Nimal Perera Family` and stores the resulting link in `patients.family_id`.

6.3 family_members

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`family_id`	BIGINT	Yes	None	Foreign key to `families.id`.
`patient_id`	BIGINT	Yes	None	Foreign key to `patients.id`.
`relationship`	VARCHAR(40)	No	None	Optional relationship label.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.

Constraints: primary key on `id`; unique on `(family_id, patient_id)`; foreign keys to `families.id` and `patients.id`. **Index:** `family_members_org_family_idx`.

7. Patient Clinical Detail Tables

7.1 patient_allergies

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`patient_id`	BIGINT	Yes	None	Foreign key to `patients.id` .
`allergy_name`	VARCHAR(120)	Yes	None	Required allergy label.
`severity`	allergy_severity	No	None	Enum constrained if present.
`is_active`	BOOLEAN	Yes	`TRUE`	Active flag.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Index: `patient_allergies_patient_idx` .

Application behavior note: allergy rows can be created inline during patient registration or later through the dedicated allergy endpoint. The allowed severity values remain `low` , `moderate` , and `high` ; values such as `medium` are not valid.

7.2 patient_conditions

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`patient_id`	BIGINT	Yes	None	Foreign key to `patients.id` .
`condition_name`	VARCHAR(180)	Yes	None	Required condition label.
`icd10_code`	VARCHAR(16)	No	None	Optional mapped code.
`status`	VARCHAR(20)	Yes	`'active'`	String status, not enum constrained.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Index: `patient_conditions_patient_idx` .

7.3 patient_vitals

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`patient_id`	BIGINT	Yes	None	Foreign key to `patients.id` .
`encounter_id`	BIGINT	No	None	Optional foreign key to `encounters.id` .
`bp_systolic`	SMALLINT	No	None	Optional systolic blood pressure.
`bp_diastolic`	SMALLINT	No	None	Optional diastolic blood pressure.

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`heart_rate`	SMALLINT	No	None	Optional heart rate.
`temperature_c`	NUMERIC(4,1)	No	None	Optional Celsius temperature.
`spo2`	SMALLINT	No	None	Optional oxygen saturation.
`recorded_at`	TIMESTAMPTZ	Yes	None	Measurement timestamp.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Index: `patient_vitals_patient_recorded_idx` on `(patient_id, recorded_at)`.

Application behavior note: vitals writes now follow a fixed typed API contract rather than generic name or value rows. The application requires at least one measurement on create, supports correction via the vitals patch route, soft deletion via the vitals delete route, validates that any supplied `encounter_id` belongs to the same patient, and can create the first vitals row during encounter bundle creation.

7.4 patient_timeline_events

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`patient_id`	BIGINT	Yes	None	Foreign key to `patients.id`.
`encounter_id`	BIGINT	No	None	Optional foreign key to `encounters.id`.
`event_date`	DATE	Yes	None	Timeline ordering date.
`title`	VARCHAR(160)	Yes	None	Required timeline title.
`description`	TEXT	No	None	Optional narrative.
`event_kind`	VARCHAR(30)	No	None	Optional category string.
`tags`	TEXT[]	No	None	Optional tag array.
`value`	VARCHAR(80)	No	None	Optional short value.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Index: `patient_timeline_events_patient_date_idx` on `(patient_id, event_date)`.

7.5 patient_history_entries

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key; introduced in V5.
`organization_id`	UUID	Yes	None	Tenant scope.
`patient_id`	BIGINT	Yes	None	Foreign key to `patients.id`.
`created_by_user_id`	BIGINT	Yes	None	Foreign key to `users.id`.
`note`	TEXT	Yes	None	Check constraint requires length between 1 and 1000.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Constraints: `patient_history_entries_note_length_chk` enforces `char_length(note) BETWEEN 1 AND 1000`. **Indexes:** `patient_history_entries_org_patient_idx`, `patient_history_entries_patient_created_idx`.

8. Scheduling and Encounter Tables

8.1 appointments

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGINT	Yes	`nextval('appointments_partitioned_id_seq')`	Sequence-backed identifier.
`organization_id`	UUID	Yes	None	Tenant scope.
`patient_id`	BIGINT	Yes	None	Foreign key to `patients.id` .
`doctor_id`	BIGINT	No	None	Optional foreign key to `users.id` .
`assistant_id`	BIGINT	No	None	Optional foreign key to `users.id` .
`scheduled_at`	TIMESTAMPTZ	Yes	None	Partition key and part of primary key.
`status`	appointment_status	Yes	None	Enum constrained.
`reason`	TEXT	No	None	Optional reason text.
`priority`	priority_level	Yes	`'normal'`	Enum constrained.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Constraints: partitioned table; primary key is composite `(id, scheduled_at)` ; foreign keys to `patients.id` and optionally `users.id` for doctor/assistant. **Indexes:** `appointments_id_idx` , `appointments_status_scheduled_idx` , `appointments_patient_idx` , `appointments_org_scheduled_idx` .

Partitioning notes: V4 creates `appointments_default` and monthly partitions such as `appointments_2026_03` .

Application behavior note: the backend now uses `appointments` for both scheduled center visits and small-clinic walk-ins. The `POST /v1/visits/start` workflow reuses an existing active row in `waiting` or `in_consultation` when available, otherwise it creates a new in-consultation visit row for the patient.

8.2 encounters

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`appointment_id`	BIGINT	Yes	None	Unique for one encounter per appointment.
`patient_id`	BIGINT	Yes	None	Foreign key to `patients.id` .
`doctor_id`	BIGINT	Yes	None	Foreign key to `users.id` .
`checked_at`	TIMESTAMPTZ	Yes	None	Encounter consult time.
`notes`	TEXT	No	None	Optional notes.
`next_visit_date`	DATE	No	None	Optional follow-up date.
`status`	VARCHAR(20)	Yes	`'completed'`	String status, not enum constrained.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.
`appointment_scheduled_at`	TIMESTAMPTZ	Yes	None	Added in V4 for composite FK to appointments.

Constraints: primary key on `id`; unique on `appointment_id`; foreign keys to `patients.id`, `users.id`, and composite `(appointment_id, appointment_scheduled_at) -> appointments(id, scheduled_at)` with `ON UPDATE CASCADE`. **Indexes:** `encounters_patient_idx`, `encounters_appointment_fk_idx`.

8.3 encounter_diagnoses

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`encounter_id`	BIGINT	Yes	None	Foreign key to `encounters.id`.
`icd10_code`	VARCHAR(16)	No	None	Optional ICD-10 code.
`diagnosis_name`	VARCHAR(255)	Yes	None	Required diagnosis label.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.

8.4 test_orders

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`encounter_id`	BIGINT	Yes	None	Foreign key to `encounters.id`.
`test_name`	VARCHAR(180)	Yes	None	Required test label.
`status`	test_order_status	Yes	`'ordered'`	Enum constrained.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.

Index: `test_orders_encounter_idx`.

9. Prescription and Dispensing Tables

9.1 prescriptions

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`encounter_id`	BIGINT	Yes	None	Foreign key to `encounters.id`.
`patient_id`	BIGINT	Yes	None	Foreign key to `patients.id`.
`doctor_id`	BIGINT	Yes	None	Foreign key to `users.id`.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Indexes: `prescriptions_patient_idx`, `prescriptions_encounter_idx`.

9.2 prescription_items

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`prescription_id`	BIGINT	Yes	None	Foreign key to `prescriptions.id`.
`drug_name`	VARCHAR(180)	Yes	None	Required medicine label.
`dose`	VARCHAR(80)	Yes	None	Required dosing string.
`frequency`	VARCHAR(80)	Yes	None	Required frequency string.
`duration`	VARCHAR(80)	No	None	Optional duration.
`quantity`	NUMERIC(12,2)	Yes	None	Required numeric quantity.
`source`	drug_source	Yes	None	Enum constrained.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Index: `prescription_items_prescription_idx`.

9.3 dispense_records

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`prescription_id`	BIGINT	Yes	None	Foreign key to `prescriptions.id`.
`assistant_id`	BIGINT	Yes	None	Foreign key to `users.id`.
`dispensed_at`	TIMESTAMPTZ	Yes	None	Dispense timestamp.

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`status`	VARCHAR(20)	Yes	`'completed'`	String status, not enum constrained.
`notes`	TEXT	No	None	Optional notes.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.

Index: `dispense_records_prescription_idx`.

10. Inventory Tables

10.1 inventory_items

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`sku`	VARCHAR(80)	No	None	Globally unique when present.
`name`	VARCHAR(180)	Yes	None	Required item name.
`generic_name`	VARCHAR(180)	No	None	Optional generic medicine name.
`category`	inventory_category	Yes	None	Enum constrained.
`subcategory`	VARCHAR(80)	No	None	Optional item subtype such as tablet or kit.
`description`	TEXT	No	None	Optional free-text description.
`dosage_form`	VARCHAR(40)	No	None	Optional dosage-form label such as tablet, syrup, or vial.
`strength`	VARCHAR(40)	No	None	Optional strength label such as 500mg.
`unit`	VARCHAR(20)	Yes	None	Required stock unit.
`dispense_unit`	VARCHAR(20)	No	None	Optional clinic-side dispense pack label added in `V16`.
`dispense_unit_size`	NUMERIC(12,2)	No	None	Optional base-unit quantity per dispense pack added in `V16`.
`purchase_unit`	VARCHAR(20)	No	None	Optional supplier-side purchase pack label added in `V16`.
`purchase_unit_size`	NUMERIC(12,2)	No	None	Optional base-unit quantity per purchase pack added in `V16`.
`route`	VARCHAR(40)	No	None	Optional administration route.
`prescription_type`	VARCHAR(20)	No	None	Optional item usage mode such as clinical, outside, or both.
`package_unit`	VARCHAR(20)	No	None	Legacy/general package label retained for compatibility.
`package_size`	NUMERIC(12,2)	No	None	Legacy/general package size retained for compatibility.
`brand_name`	VARCHAR(120)	No	None	Optional brand or trade name.
`supplier_name`	VARCHAR(120)	No	None	Optional supplier name.
`lead_time_days`	BIGINT	No	None	Optional supplier lead time in days.
`stock`	NUMERIC(12,2)	Yes	`0`	Current stock balance.
`reorder_level`	NUMERIC(12,2)	Yes	`0`	Reorder threshold.
`min_stock_level`	NUMERIC(12,2)	No	None	Optional explicit minimum stock threshold.
`max_stock_level`	NUMERIC(12,2)	No	None	Optional explicit maximum stock threshold.
`expiry_date`	DATE	No	None	Optional item or batch expiry date.
`batch_no`	VARCHAR(80)	No	None	Optional batch or lot reference.
`storage_location`	VARCHAR(120)	No	None	Optional storage location label.
`direct_dispense_allowed`	BOOLEAN	Yes	`FALSE`	Doctor direct-dispense eligibility flag.
`is_antibiotic`	BOOLEAN	Yes	`FALSE`	Antibiotic marker.
`is_controlled`	BOOLEAN	Yes	`FALSE`	Controlled-item marker.

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`is_pediatric_safe`	BOOLEAN	Yes	`FALSE`	Pediatric-safety marker.
`requires_prescription`	BOOLEAN	Yes	`TRUE`	Prescription requirement flag.
`clinic_use_only`	BOOLEAN	Yes	`FALSE`	Clinic-use-only flag.
`notes`	TEXT	No	None	Optional operational notes.
`is_active`	BOOLEAN	Yes	`TRUE`	Active flag.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Constraints: primary key on `id`; unique on `sku`. **Indexes:** `inventory_items_org_name_idx`, `inventory_items_org_generic_name_idx`, `inventory_items_org_expiry_date_idx`. **Design note:** `unit` is the base stock unit, while `dispense_unit` / `dispense_unit_size` and `purchase_unit` / `purchase_unit_size` describe conversion packs for clinic and supplier workflows.

10.2 inventory_batches

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`inventory_item_id`	BIGINT	Yes	None	Foreign key to `inventory_items.id`.
`batch_no`	VARCHAR(80)	Yes	None	Required batch or lot identifier.
`expiry_date`	DATE	No	None	Optional expiry date for the specific batch.
`quantity`	NUMERIC(12,2)	Yes	`0`	Current remaining stock in base units for this batch.
`supplier_name`	VARCHAR(120)	No	None	Optional supplier linked to the batch receipt.
`storage_location`	VARCHAR(120)	No	None	Optional storage location for this batch.
`received_at`	TIMESTAMPTZ	Yes	`NOW()`	Receipt timestamp for the batch.
`is_active`	BOOLEAN	Yes	`TRUE`	Active flag for operational use.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Creation timestamp.
`updated_at`	TIMESTAMPTZ	Yes	`NOW()`	Update timestamp.
`deleted_at`	TIMESTAMPTZ	No	None	Soft delete marker.

Indexes: `inventory_batches_item_idx`, `inventory_batches_org_expiry_idx`. **Design note:** batch rows allow one inventory item to hold multiple expiry windows or lot references while preserving item-level total stock for fast operational reads.

10.3 inventory_movements

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`inventory_item_id`	BIGINT	Yes	None	Foreign key to `inventory_items.id`.
`batch_id`	BIGINT	No	None	Optional foreign key to `inventory_batches.id` for lot-level movement tracking.
`movement_type`	inventory_movement_type	Yes	None	Enum constrained.
`reason`	VARCHAR(30)	No	None	Optional reason such as purchase, dispense, damage, expired, return, adjustment, or manual.

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`quantity`	NUMERIC(12,2)	Yes	None	Movement quantity.
`note`	TEXT	No	None	Optional operator note.
`reference_type`	VARCHAR(30)	No	None	Optional origin marker.
`reference_id`	BIGINT	No	None	Optional referenced row id.
`created_by_id`	BIGINT	Yes	None	Foreign key to `users.id`.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Immutable creation timestamp.

Index: `inventory_movements_item_idx`. **Operational note:** stock balances should be changed through movement rows instead of editing `inventory_items.stock` directly for ordinary clinic operations. From `V17` onward, movement rows can optionally reference a batch for lot-specific deductions and receipts.

11. Audit and Operational Tables

11.1 audit_logs

COLUMN	TYPE	REQUIRED	DEFAULT / GENERATED	CONSTRAINTS AND NOTES
`id`	BIGSERIAL	Yes	Sequence generated	Part of composite primary key.
`organization_id`	UUID	Yes	None	Tenant scope.
`actor_user_id`	BIGINT	No	None	Optional foreign key to `users.id`.
`entity_type`	VARCHAR(60)	Yes	None	Target entity category.
`entity_id`	BIGINT	No	None	Optional target row id.
`action`	VARCHAR(30)	Yes	None	Action verb.
`ip`	INET	No	None	Optional client IP.
`user_agent`	TEXT	No	None	Optional client string.
`request_id`	UUID	No	None	Optional correlation id.
`payload`	JSONB	No	None	Optional structured payload.
`created_at`	TIMESTAMPTZ	Yes	`NOW()`	Partition key and part of composite primary key.

Constraints: partitioned by range on `created_at`; composite primary key `(id, created_at)`; foreign key `actor_user_id -> users.id`. **Indexes:** `audit_logs_created_idx`, `audit_logs_entity_idx`. **Partitions:** `audit_logs_default`, `audit_logs_2026_03`, `audit_logs_2026_04`.

12. Constraint and Validation Highlights

- **NOT NULL enforcement:** identity, relationship, and workflow-critical fields are mandatory in the schema, including `patients.first_name`, `patients.last_name`, `patients.full_name`, `patients.dob`, and `patients.age`.
- **Unique keys:** used for deduplication and natural identifiers such as user email per organization, patient NIC per organization, token id, family code, and SKU.
- **Foreign keys:** enforce patient, appointment, encounter, prescription, inventory, and actor relationships.
- **Enum validation:** several status and category fields are strictly database-constrained enums.
- **Check constraints:** explicit length validation exists on `patient_history_entries.note`.
- **Generated columns:** `patients.full_name` is database-generated and stored.
- **Partitioning:** `appointments` and `audit_logs` are partitioned for scale and maintenance.

13. Source Files

- `infra/flyway/sql/V1__init.sql`
- `infra/flyway/sql/V4__partition_appointments.sql`

- `infra/flyway/sql/V5__add_patient_history.sql`
- `infra/flyway/sql/V6__phase4_observability_security.sql`
- `infra/flyway/sql/V7__enforce_patient_required_fields.sql`
- `infra/flyway/sql/V8__seed_additional_patients.sql`
- `infra/flyway/sql/V9__add_user_extra_permissions.sql`
- `infra/flyway/sql/V11__add_doctor_workflow_mode.sql`
- `infra/flyway/sql/V12__add_user_roles_and_active_role.sql`
- `infra/flyway/sql/V13__add_analytics_workflow_timestamps.sql`
- `infra/flyway/sql/V14__expand_inventory_management_and_alerting_fields.sql`
- `infra/flyway/sql/V15__add_doctor_facing_inventory_fields.sql`
- `infra/flyway/sql/V16__add_inventory_packaging_conversion_fields.sql`
- `infra/flyway/sql/V17__add_inventory_batches_and_reporting_support.sql`

This dictionary is intended for review, onboarding, QA, and client walkthroughs. When the schema changes, update this document alongside the relevant Flyway migration. Current inventory coverage includes alerts metadata, doctor-facing inventory fields, packaging-conversion fields, and batch-level inventory support through `V17`.